

Introduction

Instruction set architecture blueprints the programmer's view of the machine using words and tables. It describes:

- data processing instructions provided by the machine like **add**, **sub**, **and**, **eor**, and **mul**,
- memory load-store instructions provided by the machine like **ldr**, and **str**,
- branch instructions provided by the machine to control the program counter,
- the names and sizes of the data and control registers,
- the size and endianness of data memory, and
- the binary encoding formats for each instruction.

Microarchitecture is a microchip implementation of an instruction set architecture built by interconnecting digital logic components to form a working circuit. Multiple microarchitectures are possible because design engineers can choose different components or different interconnections when implementing the machine. In the end, however, any microarchitecture must implement the instruction set architecture.

One organizational decision that leads to different microarchitectures is the number of clock periods used per instruction. Three common strategies are **single cycle**, **multicycle**, and **pipelined**.

- Single cycle processors use one clock-period per instruction and the clock-period is set by the delay of the slowest instruction. The disadvantage is faster instructions cannot execute more quickly. The advantage is straightforward control circuitry.
- Multicycle processors use multiple clock periods per instruction and each instruction uses the minimum number of clock periods required for its execution. This allows an instruction to avoid delay in circuit stages that are not used to implement the instruction. The advantage is speed. The disadvantage is more complex control because a finite state machine controller must be built to coordinate control signals across multiple clock periods.
- Pipelined processors exploit instruction level parallelism to allow multiple instructions to be in stages of execution at the same time. This is accomplished by adding state registers between the fetch, decode, execute, memory access, and register write circuit stages.

The **CPE1510 single cycle processor** implements a subset of the ARMv4 instructions. Students are required to:

- **Design** VHDL implementations of the ARMv4 data path components and the data path controller.
- **Interconnect** the data path components into a single cycle processor schematic.
- **Simulate** the single cycle processor using a basic test program.

Laboratory Objectives

- Integrate ARMv4 instruction set knowledge with prerequisite VHDL design techniques.
- Synthesize a microarchitecture by applying skills in computer aided design.
- Critique a design using timing-based simulation results.

Our Design

The **textbook design** closely follows the ARMv4 instruction set architecture. ARMv4 reserves register R15 as the program counter (PC) and any read from register R15 must return the value PC+8. In the textbook microarchitecture, the authors form PC+8 using an adder and route the result to the register file through an input called R15. Branch instructions use the ALU to finish forming the branch address as $(PC+8)_{SRC1} + (\text{SignExtImm24} \ll 2)_{SRC2}$. The textbook also presents a controller divided into three parts. The first part of the controller is a four-bit register storing the conditional code nibble CVNZ. The second part of the controller is a decoder that generates control signals after examining the opcode, I-bit, command field, and S-bit. Finally, the third part of the controller is a set of conditional execution logic that allows or prevents register file and memory updates during conditional execution.

An **alternate design** is presented in this laboratory. The CPE1510 ARMv4 design calculates the branch address without routing through the register file or ALU. Also, the instruction set implemented by the controller is limited to unconditional data processing instructions with and without S suffix, offset mode LDR and STR instructions, unconditional branches B and BL, and conditional branches BEQ, and BNE. This limited instruction set results in a simpler controller that can be implemented quite cleanly using VHDL when-else statements for each control signal.

Engineering is both art and science. In this course, you have studied the ARMv4 instruction set architecture and basic assembly language programming, learning the computer **science** behind this numeric processor. You can see the **art** of engineering by studying both the textbook design and this laboratory design. Both designs achieve the desired goal of allowing ARMv4 machine code instructions to execute. The choices made in this alternate laboratory design prevent the processor from executing all ARM instructions that would be expected to work if presented as machine code instructions in a commercial processor. For example, the LDR and STR instructions are greatly reduced in functionality. And no conditional arithmetic, such as ADDEQ, SUBLT, or RSBNE are implemented.

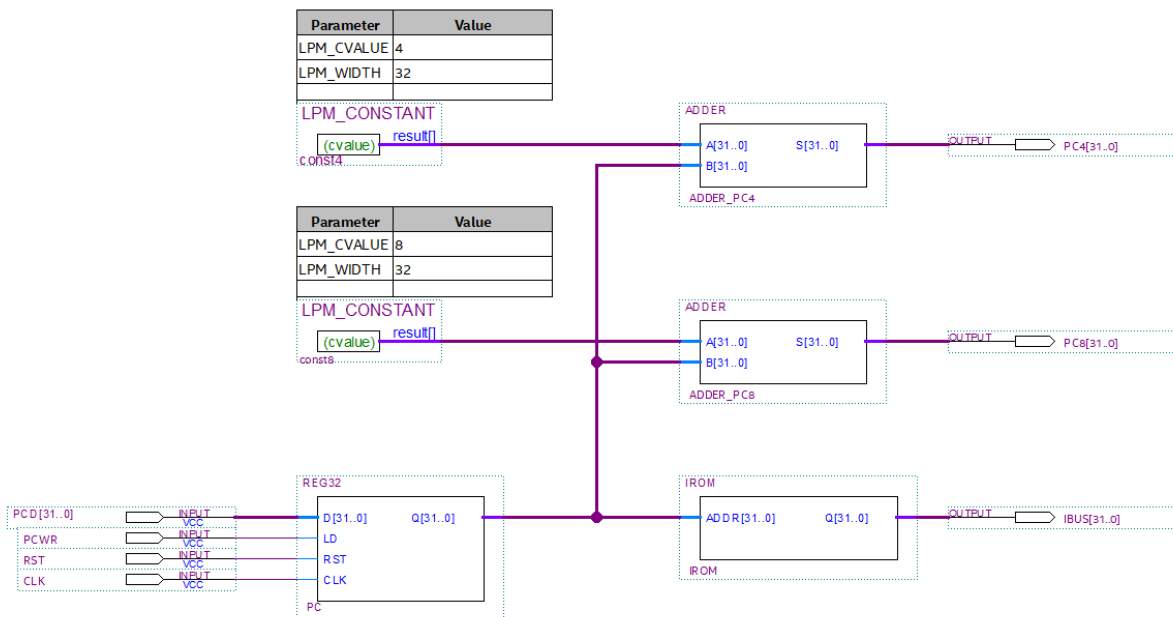
Assignment

- **Create** a Quartus project called scp.
- **Complete** the VHDL skeleton-file descriptions of the ARMv4 data path components.
- **Interconnect** data path components into the top-level **schematic** called scp.bdf.
- **Simulate** the top-level schematic and **verify** that the program functions.
- **Demonstrate** your working single cycle processor to the instructor.

References

- A reference PDF has been provided showing the complete schematic.
- VHDL skeleton files have been provided for all components.
- Summaries of the circuit stages follow in this document.
- Simulation guidance is provided at the end of this document.

Stage Summary: Fetch



Fetch provides the machine code from the instruction memory and pre-calculates program counter values.

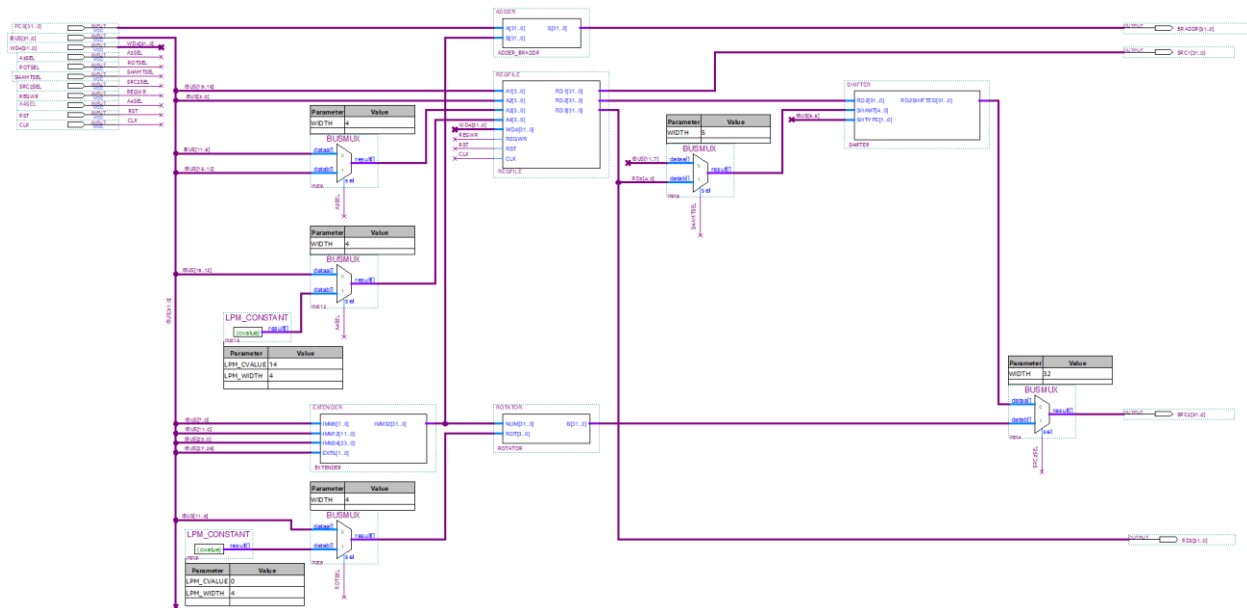
- Instruction memories are usually flash-ROMs in the microchips used in embedded systems. ROM memories output the stored value requested by the address input but never write new data in user mode. Thus, this component has no load control signal or sample clock. In addition, large memories generally do not include reset signals because it would take too much time to reset millions or billions of locations.

The ARMv4 control flow equations are $PC \leftarrow PC+4$ and $PC \leftarrow (PC+8)+(SExtImm \ll 2)$. The fetch circuit uses LPM_CONSTANT components and VHDL adders to calculate PC+4 and PC+8. These calculations occur at the same time because of two separate adders. Architects say that these calculations occur **in parallel**.

Use the VHDL skeleton files from this laboratory to complete:

- A rising-edge triggered register with active-low **asynchronous reset** and active-low **synchronous load** ([reg32.vhd](#))
- An adder using unsigned types and the + operator from IEEE library [ieee.numeric_std](#) ([adder.vhd](#))
- The fetch stage schematic ([fetch.bdf](#)) with components given the unique names shown in the figure.

Stage Summary: Decode



Decode provides the data used by the execute stage.

- Data source SRC1 is always the register requested by the rn address field: R[Rn]
- Register mode data processing requires SRC2 to be R[Rm] shifted by an unsigned immediate shift amount.
- Register-shifted register mode data processing requires SRC2 to be R[Rm] shifted by unsigned R[Rs].
- Immediate mode data processing requires SRC2 to be a 32-bit unsigned immediate.
- Memory processing instructions require SRC2 to be a 32-bit unsigned immediate.

The decode circuit in the CPE1510 single cycle processor also calculates the branch address.

- Branch instructions require a 32-bit signed immediate.
- The branch address is calculated as: $BrAddr = (PC+8) + (SExtImm32 \ll 2)$

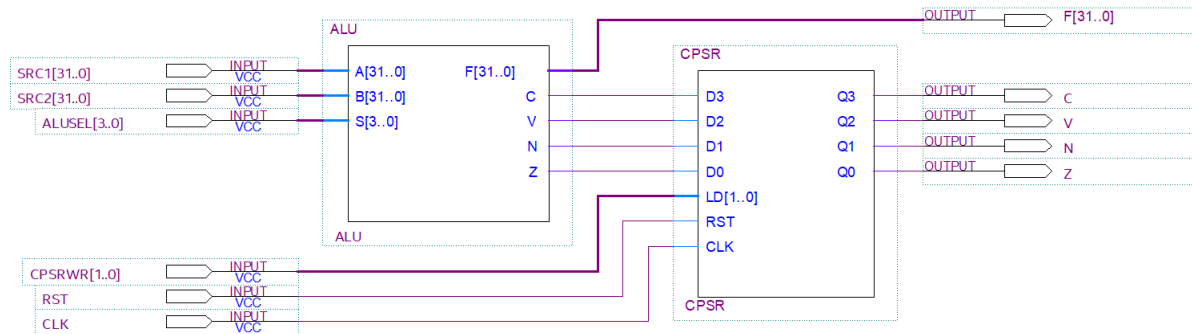
Use the VHDL skeleton files from this laboratory to complete:

- A rising-edge triggered register file with active-low **synchronous reset** ([regfile.vhd](#)).
- An immediate extender ([extender.vhd](#)).
- An immediate rotator ([rotate.vhd](#)).
- A shifter ([shifter.vhd](#)).
- The decode stage schematic ([decode.bdf](#)) with components renamed from **inst** names as shown.
- The schematic shows the vertical instruction bus labeled with IBUS[31..0].

Quartus includes a 2:1 bus multiplexer.

- Type **busmux** when searching for the symbol.
- Set the **width** parameter of each busmux to 4-bits because they chose between 4-bit wide bitfields.

Stage Summary: Execute



Execute completes calculates a data processing result or the data memory address.

- Control signal ALUSEL selects the ALU function.
- Control signal CPSRWR determines which ALU flag signals are sampled when the S-bit is true.

IF S = 1 THEN		
Instruction Type	Instructions	CPSR Samples and Stores
Arithmetic	ADDS, RSBS, SUBS	CVNZ
Compare	CMN, CMP	CVNZ
Logic	ANDS, BICS, EORS, ORRS	CNZ
Shifts	ASRS, LSLs, LSRs, RORS	CNZ
Moves	MOVS, MVNS	CNZ

Use the VHDL skeleton files from this laboratory to complete:

- The arithmetic and logic unit ([alu.vhd](#)).
- A rising-edge triggered CPSR with active-low **synchronous reset** and **synchronous load** ([cpsr.vhd](#)).
- The execute stage schematic ([execute.bdf](#)) with components given the unique names shown in the figure.

Circuit Summary: Control

The control unit implements a truth table that uses the fields of the instruction bus and the ALU flag bits as inputs.

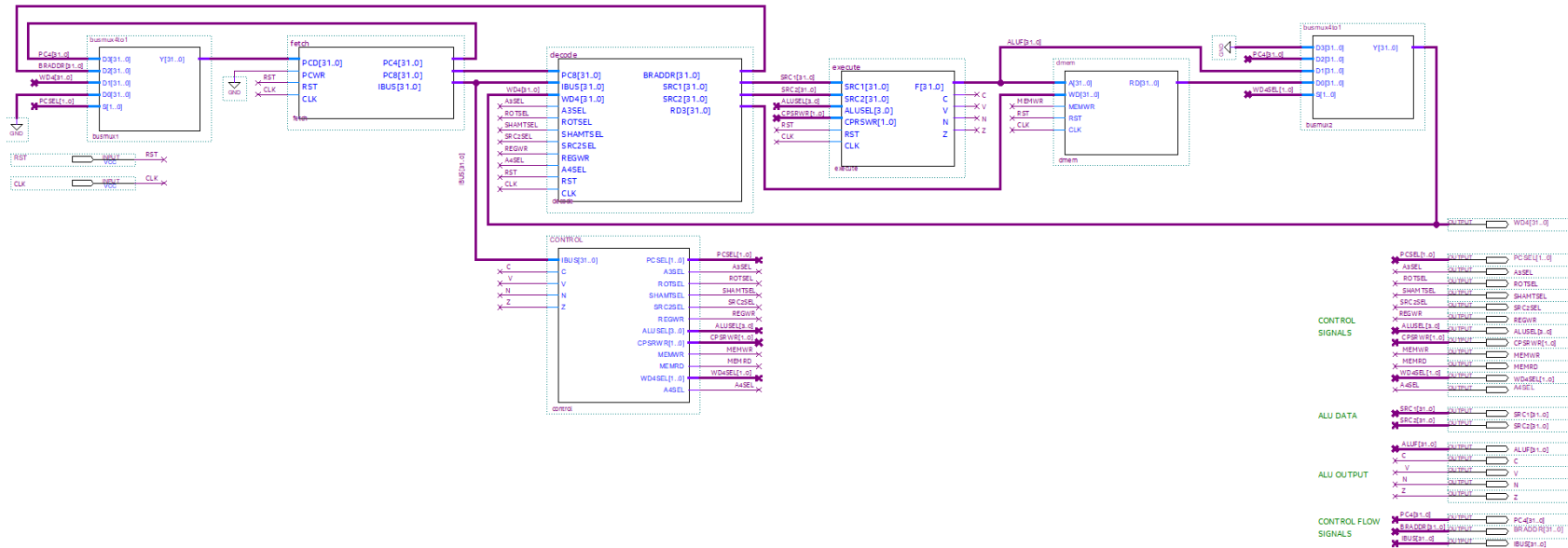
- Outputs choose multiplexer routes.
- Outputs activate write behaviors in the register file, data memory, and CPSR.
- Outputs select the ALU function.

Use the VHDL skeleton file from this laboratory and the Excel truth table started in lecture to implement:

- A control circuit ([control.vhd](#)) to control the flow of data and state changes in the registers.
- The control circuit is a combinational logic circuit.
- The control circuit is implemented in VHDL as a set of when-else statements, one for each output.
- The control circuit must implement these instructions from the front page of the CPE 1510 ARMv4 Basic Instruction Set Architecture Quick Reference card.

Data Processing			
Instructions	Register Mode	Shifted Register Mode	Immediate Mode
add, adds	☒	☒	☒
and, ands	☒	☒	☒
asl, asls	☒	☒	☒
asr, asrs	☒	☒	☒
bic, bics	☒	☒	☒
cmn	☒	☒	☒
cmp	☒	☒	☒
eor, eors	☒	☒	☒
lsl, lsls	☒	☒	☒
lsr, lsrs	☒	☒	☒
mov, movs	☒	☒	☒
mvn, mvns	☒	☒	☒
orr, orrs	☒	☒	☒
rsb, rsbs	☒	☒	☒
sub, subs	☒	☒	☒
Memory			
Instruction	Register Mode	Register $\pm$ Offset	Immediate $\pm$ Offset
ldr	☒	☒	☒
str	☒	☒	☒
Branch			
Instruction	Immediate Mode		
bal	☒		
beq	☒		
bne	☒		
bl	☒		

Top Level Design



The processor begins executing the program stored in the instruction ROM after reset. One instruction executes per clock period. **Complete** the single cycle processor schematic ([scp.bdf](#)). Give unique names to the components as shown in the figure.

- Data voltages flow back to the register file along write back signal WD4.
- Program counter voltages flow back to the program counter multiplexer on signals PC4, BRADDR, and WD4 (which is used for MOV PC,LR).
- Internal busses, such as SRC1 and SRC2, are labeled so that they can be attached to output ports for visualization during simulation.
- Control signals are attached to output ports for visualization during simulation.
- Name association is used to interconnect signals without drawing wires.

Simulation

Simulating a large circuit is an art that develops through practice over multiple years.

- **Basic processor simulation** includes RST, CLK, and WD4 because they are sufficient to check data flow.
- **Good simulation** adds output pins to visualize control signals, SRC1, SRC2, ALUF, CVNZ, PC4 and BrAddr.

When thinking about simulating any circuit, begin by identifying the expected behavior. In the case of the single-cycle processor in this laboratory, it is running a program stored in instruction ROM written by the instructors. The program calculates the sum of the first 10 numbers using a do-while loop. The sum of the first ten numbers is decimal 55. If the sum is greater than 32, the program writes the value true = "1" at memory address four (4) using STR. If the sum is not greater than 32, the program writes the value false = "0" at memory address four. It then enters an infinite loop of two instructions: the first reads memory address four using LDR and the second branches back to the LDR. Thus, simulation to show operation should clearly show:

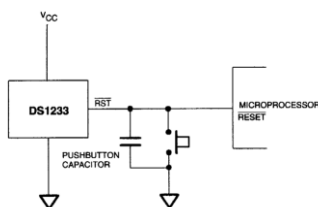
- A do-while loop and the associated PCSRC changes proving BRADDR is routed to the PC.
- A correct sum calculation.
- An ending infinite loop of LDR followed by BAL. Each time LDR executes, a one (1) should be visible on WD4 because the program earlier determined 55 to be greater than 32 and stored one (1) to memory.

The first step in simulation is adding appropriate input, output, and internal test signals. Make a new University Program Waveform file in Quartus. One organization of the waveforms on the simulation might be:

- RST and CLK clear state registers and advance the program counter. Place at the top of the simulation.
- WD4, PC4, and BRADDR are the data flow and control flow outputs. Place them next in the simulation.
- The instruction bus shows the machine code executing in each clock period. Add it next.
- Control signals are critical to operation. They should be visualized to help identify any design errors. Place them next in the simulation in the order given in the controller design truth table.
- ALU inputs are the data that drive calculation. Place SRC1, SRC2, and ALUF next in the simulation.
- ALU outputs are useful for debugging WD4 result values. Place ALUF, C, V, N, and Z next in the simulation.

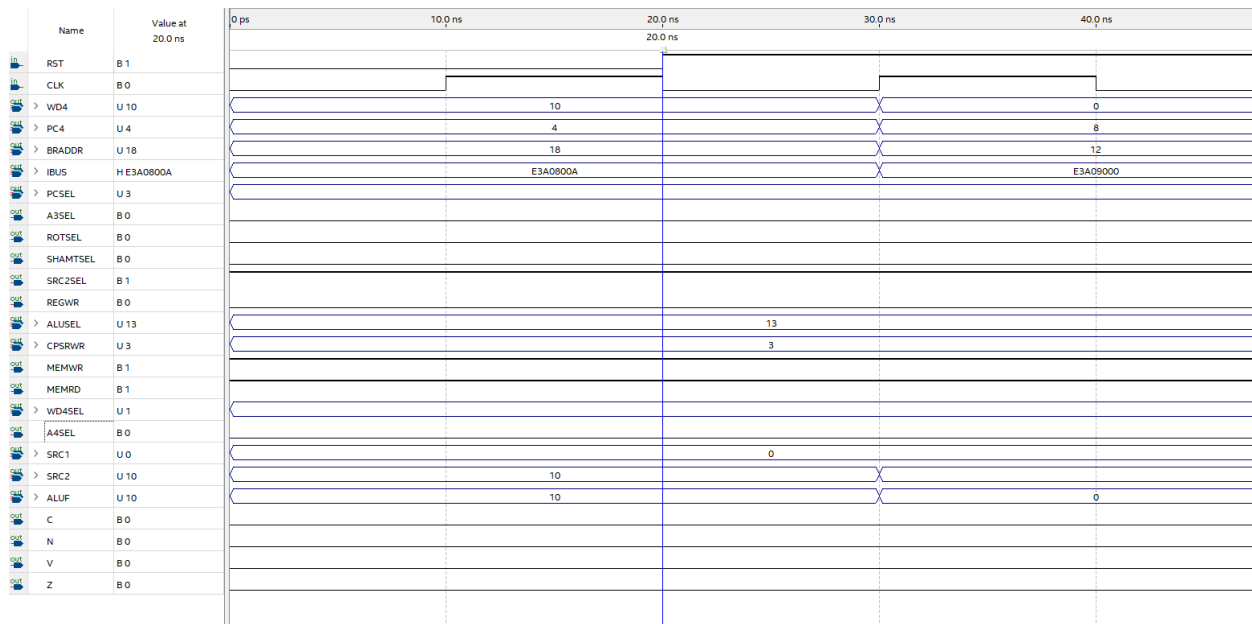
The second step is thinking about how input signals to the circuit must be controlled by the simulation.

- Extend simulation time to 2 us in the waveform editor using Edit → Set End Time.
- There are only two input signals: reset (RST) and clock (CLK).
- Add a 10ns clock waveform. For this lab, frequency and period are not important so 10ns is fine.
- On motherboards, an EconoReset and pushbutton provide system power-on and user pushbutton reset. (<https://datasheets.maximintegrated.com/en/ds/DS1233.pdf>). You do not build this circuit in Quartus.



- Simulate the EconoReset by writing logic-1 on the RST signal through simulation time. Then write logic-0 on the RST signal from the start of simulation time to the first falling edge of the clock.

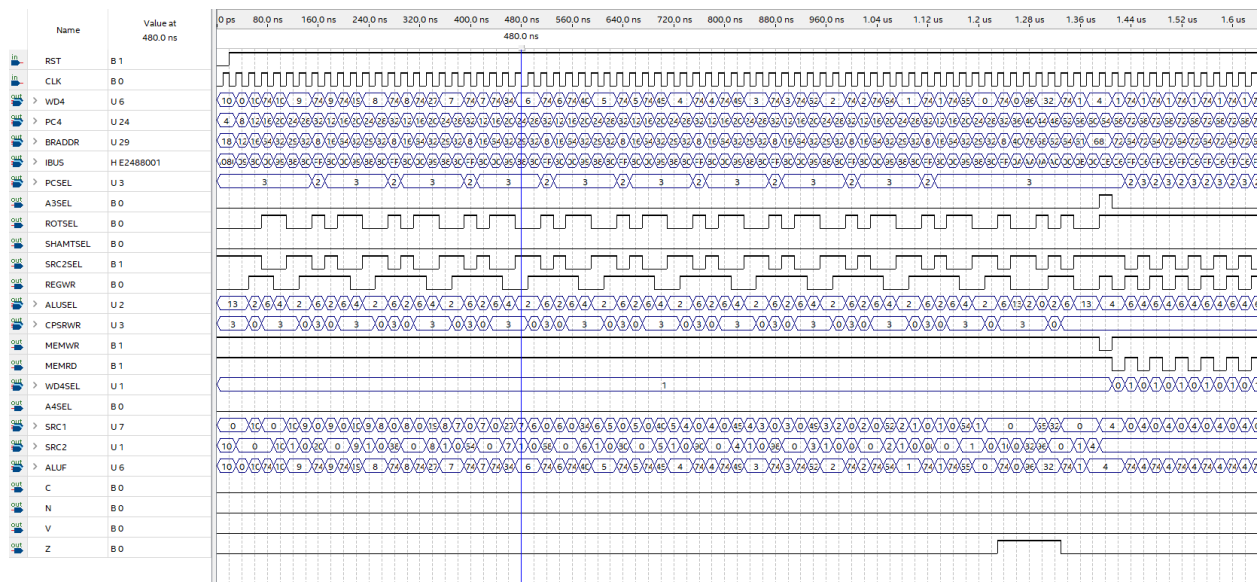
Simulation diagrams follow to help verify operation of your processor. Read the comments to learn simulation skill.



The initial simulation step should examine the first executing instruction to see if the control signals are routing data correctly and calculating the correct result.

- The first instruction in the program is MOV R8, #10 at memory location 0x0000_0000.
- Placing the time bar at the end of the reset event allows verification of all control signals, data flow, and control flow for the first instruction.
- The current PC, the PC in this clock period, is at 0x0000_0000 because PC4 reports 0x0000_0004.
- The BRADDR is not relevant for a MOV instruction and can be ignored.
- The IBUS shows the machine code shown in *from.vhd* for address 0x0000_0000.
- Control signal PCSEL correctly select PC+4 as the next PC value; seen by PCSEL = route 3.
- Control signal A3SEL is a don't care for the MOV instruction and can be ignored.
- Control signal ROTSEL correctly routes the **rotate** machine code field; seen by ROTSEL = route 0.
- Control signal SRC2SEL correctly routes the **extended immediate** as SRC2; seen by PCSEL=1.
- Control signal REGWR is active because MOV should store its result in the register file.
- Control signal ALUSEL correctly selects MOV; seen by ALUSEL = 13.
- Control signal CPSRWR correctly selects **hold** and do not sample CVNZ; seen by CPSRWR = 3.
- Active-low control signals MEMWR and MWMRD are correctly not active; this is not LDR or STR.
- Control signal WD4SEL correctly routes the ALUF output as the data result; seen by WD4SEL=1.
- Control signal A4SEL correctly routes the **rd** machine code field as the destination; seen by A4SEL=0.
- The SRC1 value is not relevant for a MOV instruction and can be ignored.
- The SRC2 value is the correct extended immediate value 10.
- The ALUF result of MOV is the correct output of the value 10.
- The ALU conditional signals to not change; MOV does not sample and change the condition signals.
- The WD4 data that is stored in the destination register in the register file is the correct value 10.

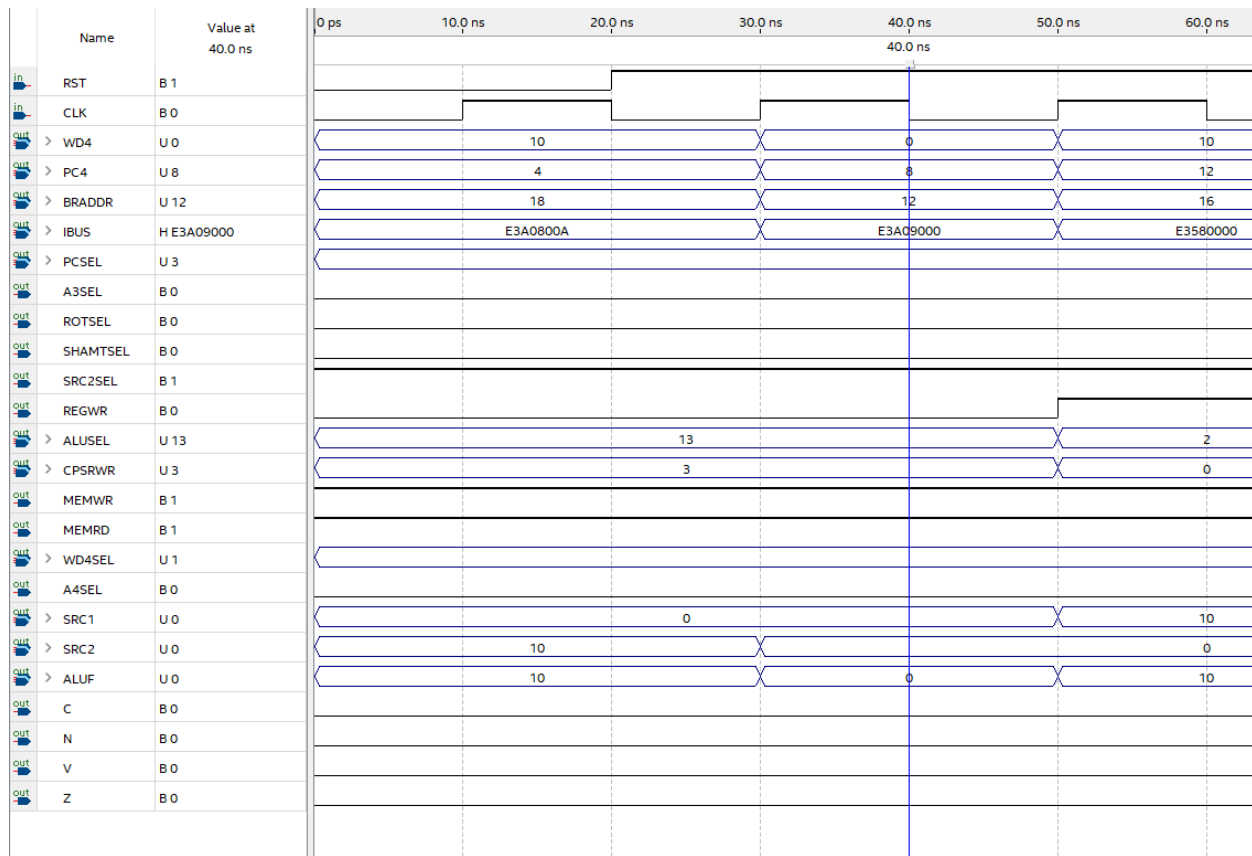
This signal-by-signal examination should be done for each instruction in the program.



A high-level simulation view can allow quick examination for loop behavior. Here, significant energy transfer is seen on the data flow and program flow signals.

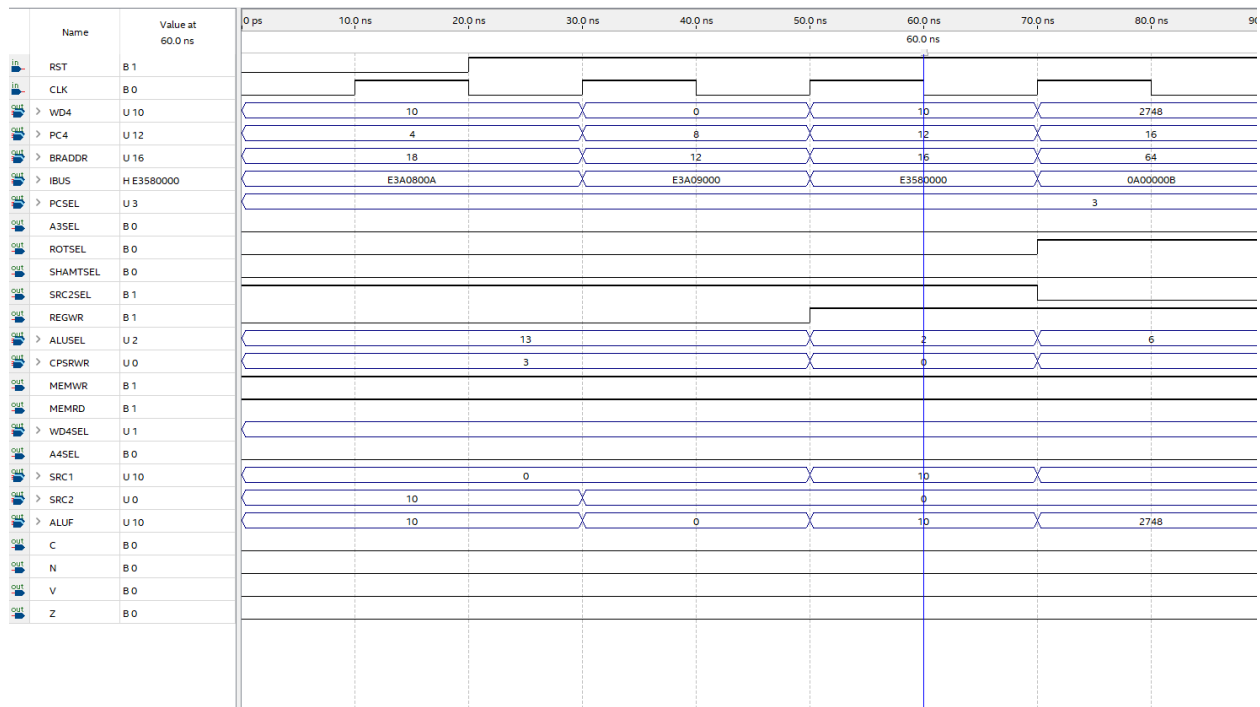
- A loop is clearly visible on the WD4 output. The loop is a down-loop counting from 10 to 0. The time bar is placed just to the right of the time where WD4 becomes 6.
- Selection of PC = BrAddr is visible as PCSEL=2 three times to the left of the time bar.
- Compare instructions are visible on the CPSRWR control signal. When compare is executing, CPSRWR=0 and REGWR=1.
- The end of the loop is also visible to the right of 1.2us when CMP discovers that the count register has reached zero. At this point the Z bit in the condition code register becomes true.
- There is only one STR instruction in the program and thus MEMWR=0 for only one clock period to the right of 1.3us.
- The end of the simulation shows a two-instruction infinite loop as PCSEL oscillates between the values 2 and 3, and MEMRD begins activating each time the infinite loop LDR executes.
- While difficult to see, the LDR WD4 value is 1 – as expected because the program should have stored a 1 into memory address 4 when it determined that 55 was greater than 32.

This simulation looks very good. It appears that the processor is working. Additional verification requires zooming in to look at key values.

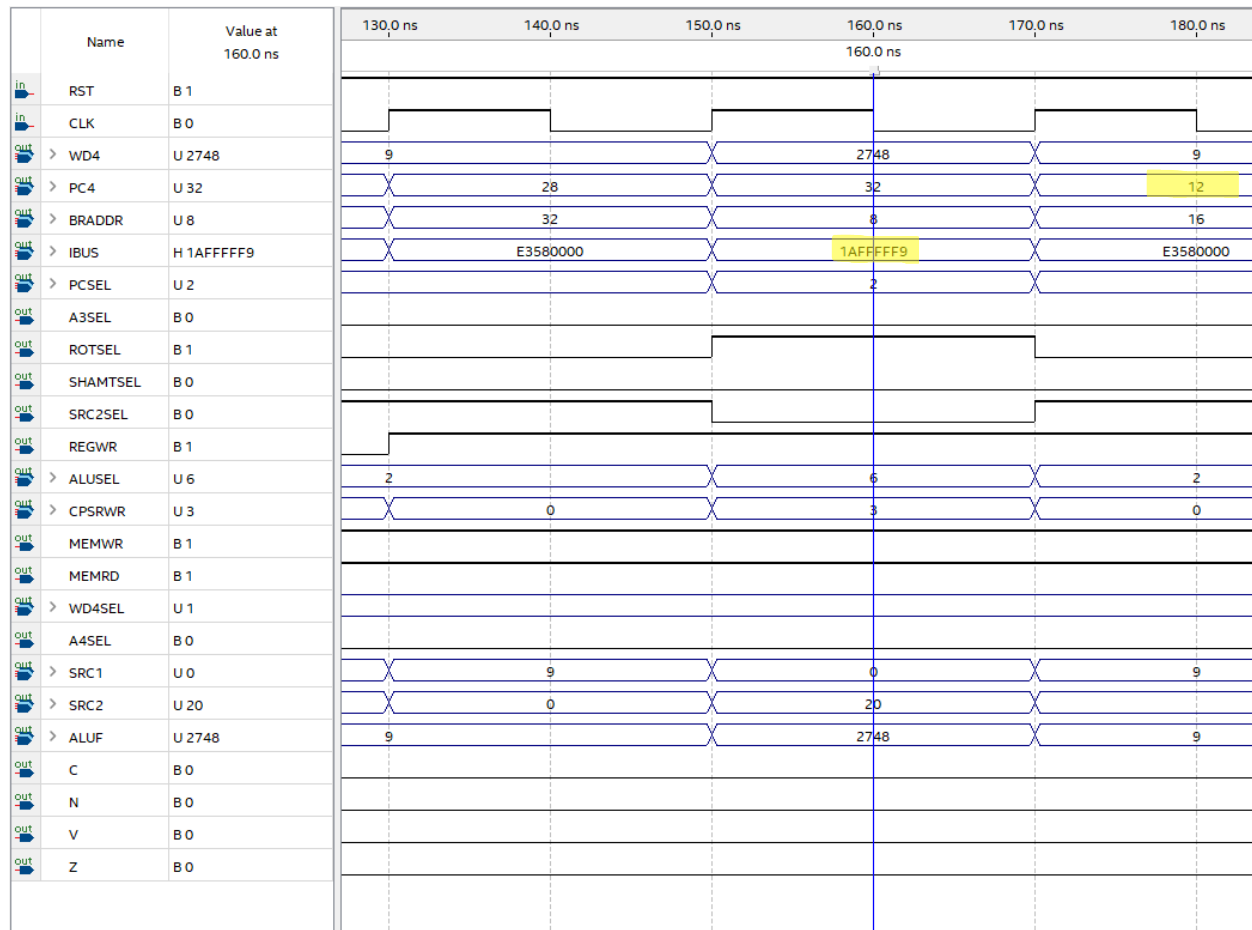


- Returning to the second instruction, simulation shows MOV operating correctly: WD4=0 for MOV R9,#0.
- Note how the verification cursor is placed right in the middle of the instruction clock period.

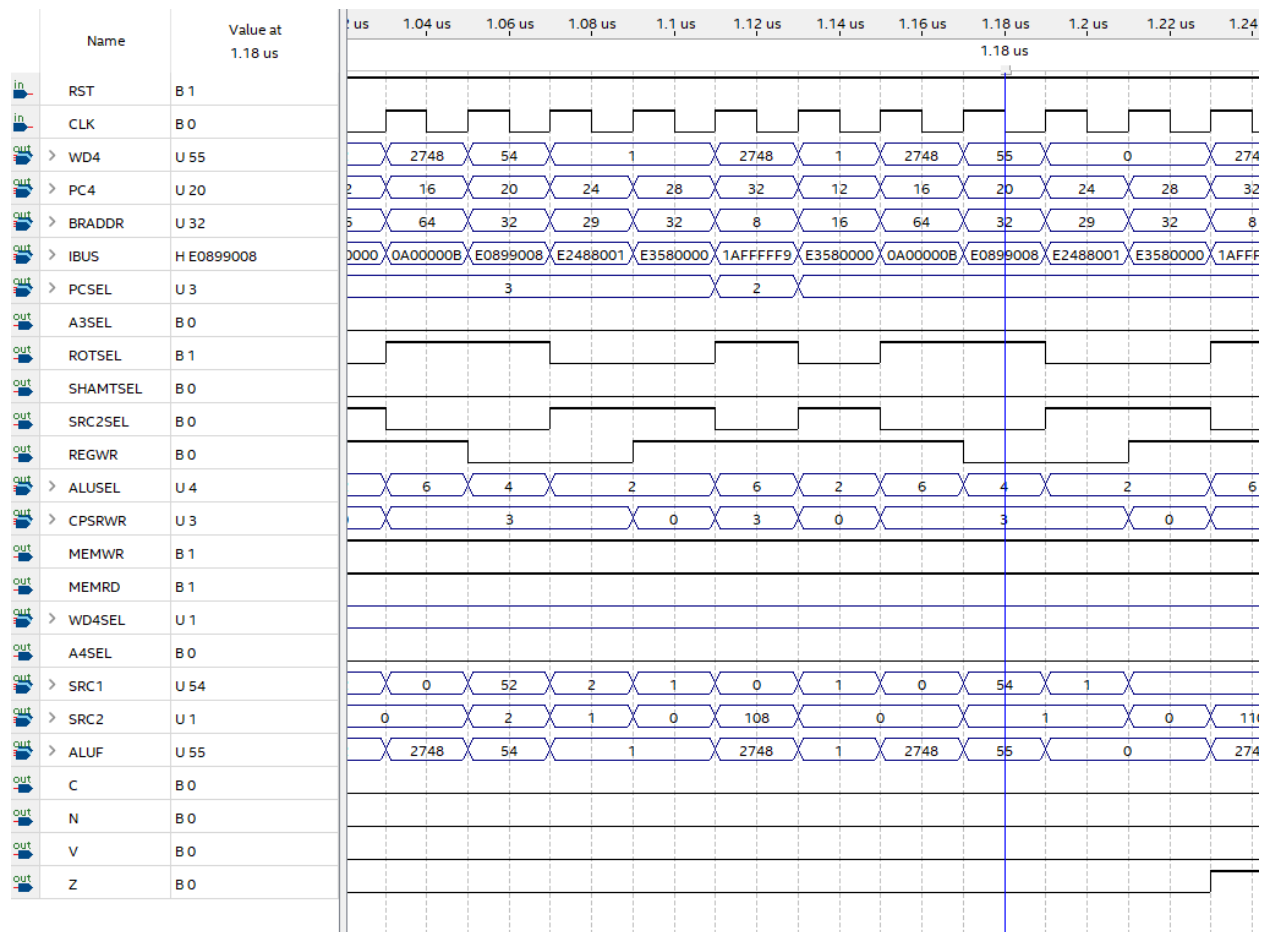
Every instruction in the program can be verified using this zoom-in technique to compare the instruction to its control signals, data sources, data result, and control flow values.



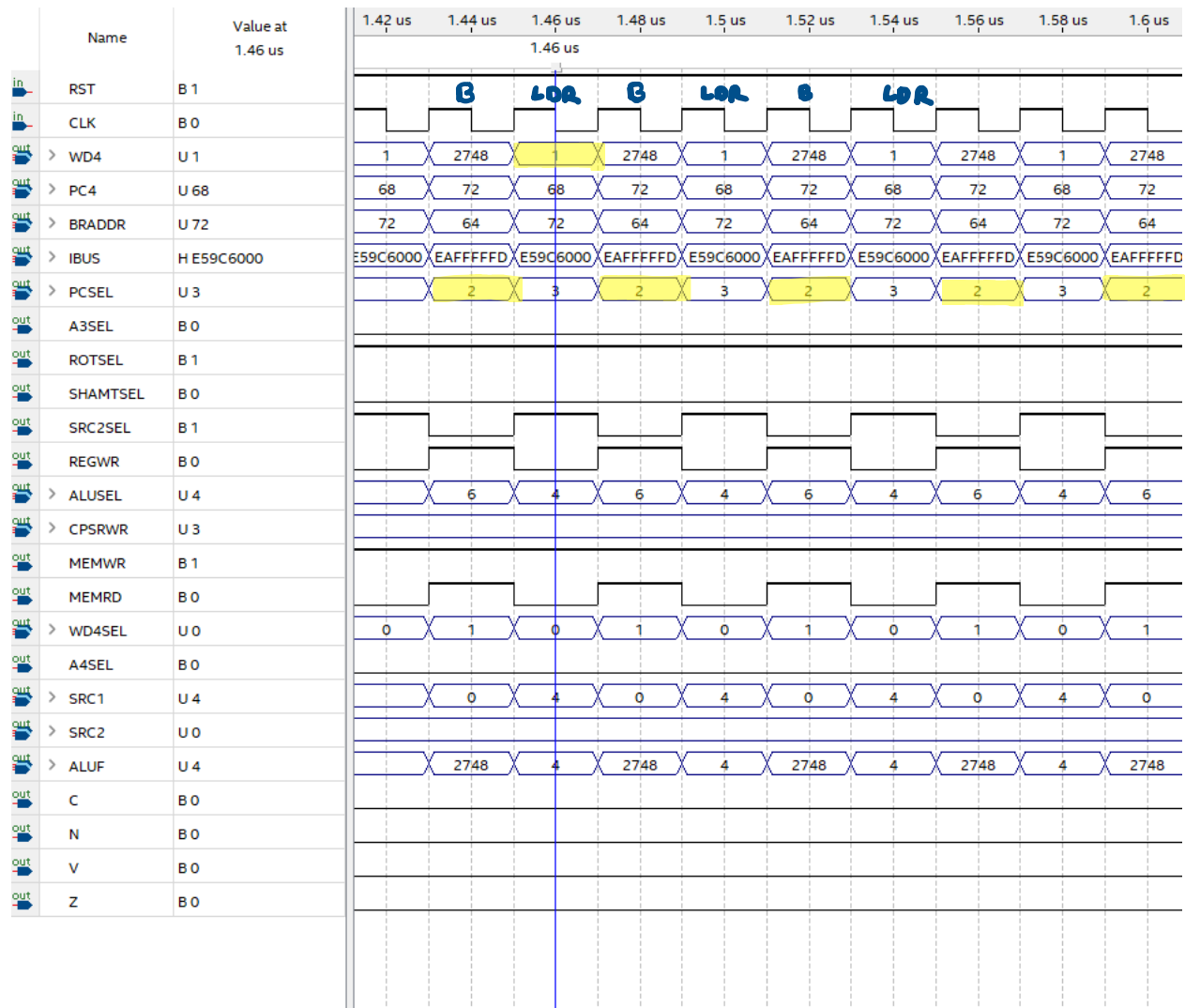
- The third instruction is operating correctly. The instruction is CMP R8, #0. CMP should have ALUSEL=2 because it subtracts SRC1 and SRC2.
- The ALU is calculating $10 - 0 = 10$. SRC1 is seen as 10, SRC2 as 0, and the subtraction result of 10 is seen on WD4 because WD4SEL=1.
- CPSRWR=0, REGWR=1, MEMWR=1 shows the CPSR changing state value for CMP, while the register file and data memory do not change value. This is correct because the subtraction is not stored in CMP.



- The eighth instruction at memory address 0x0000_001C (decimal 28) is operating correctly. This is a BNE instruction at the bottom of the do-while loop. In the simulation, you see it at address 32 because you are looking at PC+4.
- The condition code nibble in the machine code is 1. This is indeed the BNE instruction.
- PCSEL=2 because BNE **takes the branch** if the previous compare did not find equal numbers (Z=0).
- The WD4 data is not relevant for branch and can be ignored. It is not relevant because the BRADDR is calculated by a separate circuit and does not use the ALU. WD4 shows as a “garbage” value because the ALU **is calculating something** that just doesn’t get used in the instruction. It is a random number based on the don’t care values for the control signals of the ALU circuit.
- The highlighted IBUS and PC4 values show the branch occurring. The BRADDR of 8 is written into the PC on the next rising edge of the clock – causing PC4 to show 8+4 = 12.



- The end of the loop should show the correct answer calculated for the sum of the first 10 numbers. The time bar verifies SUM=55 on WD3 during the final ADD instruction in the loop.
- The compare that occurs two instructions later (CPSRWR=0) causes the update of the CVNZ bits and after this final time through the loop, the counter has reached 0 and Z becomes 1.



- One final check of the infinite loop verifies that the logic-1 **has been stored in memory** and now is being retrieved successfully from memory using LDR.
- The infinite loop is clearly visible when PCSEL routes PC = BrAddr.
- The infinite loop is also clearly visible on MEMRD signal patterns as it goes active-low for each time LDR.

Demonstration and Submission

- Well-commented VHDL code is expected.
- Create Quartus Project archives using Project → Archive Project.
- Archive your Quartus project with **your name**:

lastname-firstname-scp.qar

For example, Dr. Livingston's file would be named livingston-adam-scp.qar

- Upload your Quartus archive by the **due date** to this folder

[Dr. Livingston File Submission](#)

so that Dr. Livingston can examine your work on his machine.